Missingness Analysis: MCAR Robustness

STAT778 Project

2026-04-24

Contents

What this Rmd does

Analyzes the MCAR missingness study: same Poisson(5) baseline as K13-A default, fitted under five missingness rates: 0%, 10%, 20%, 30%, 50%.

Question. How does estimator accuracy degrade as observations are randomly missing? Does GQT degrade at the same rate as GHK, or does one of them cope better with incomplete data?

Load and tidy

```
source("code/R/logging.R")
source("code/R/io.R")
source("code/R/fit_core.R")

DATASET <- "MISSING"

ids <- list_scenarios(DATASET, dir = "data/generated")
fits <- lapply(ids, function(id) load_fit_result(DATASET, id))
names(fits) <- ids

length(fits)   # should be 5
```

```
## [1] 5
```

Extract rates from each scenario's config

```
# missing_rate lives in spec$missing_rate per the builder. The saved
# scenario config preserves it verbatim.
# We also double-check by measuring the actual NA fraction in the
```

```r
# scenario file.
rate_info <- do.call(rbind, lapply(fits, function(f) {
  cfg <- f$scenario_config
  data.frame(
    scenario_id    = cfg$scenario_id,
    spec_rate      = cfg$missing_rate,
    stringsAsFactors = FALSE
  )
}))
rate_info
```

```
##              scenario_id spec_rate
## mcar_rate0   mcar_rate0       0.0
## mcar_rate10 mcar_rate10       0.1
## mcar_rate20 mcar_rate20       0.2
## mcar_rate30 mcar_rate30       0.3
## mcar_rate50 mcar_rate50       0.5
```

Tidy to long format

```r
tidy_one <- function(f) {
  r <- f$results
  cfg <- f$scenario_config

  rows <- list()
  for (i in seq_len(nrow(r))) {
    if (!r$converged[i]) {
      rows[[length(rows) + 1]] <- data.frame(
        scenario_id = f$scenario_id,
        missing_rate = cfg$missing_rate,
        replicate   = r$replicate[i],
        method      = r$method[i],
        converged   = FALSE,
        runtime_sec = r$runtime_sec[i],
        n_used      = r$n_used[i],
        parameter   = NA_character_,
        estimate    = NA_real_,
        stringsAsFactors = FALSE
      )
      next
    }
    est <- r$estimates[[i]]
    if (is.null(est) || length(est) == 0) next
    for (pname in names(est)) {
      rows[[length(rows) + 1]] <- data.frame(
        scenario_id = f$scenario_id,
        missing_rate = cfg$missing_rate,
        replicate   = r$replicate[i],
        method      = r$method[i],
        converged   = TRUE,
        runtime_sec = r$runtime_sec[i],
        n_used      = r$n_used[i],
        parameter   = pname,
        estimate    = as.numeric(est[[pname]]),
```

```
        stringsAsFactors = FALSE
      )
    }
  }
  do.call(rbind, rows)
}

long <- do.call(rbind, lapply(fits, tidy_one))

# Attach truth (log-lambda for Intercept, range for range).
# Every scenario shares the same generating params, so truth is fixed.
long$truth <- NA_real_
long$truth[long$parameter == "Intercept"] <- log(5)
long$truth[long$parameter == "range"]     <- 0.3
long$abs_err <- abs(long$estimate - long$truth)
long$err     <- long$estimate - long$truth

nrow(long)
```

```
## [1] 4000
```

```
head(long)
```

```
##              scenario_id missing_rate replicate method converged runtime_sec
## mcar_rate0.1  mcar_rate0            0         1    GHK      TRUE   0.7393694
## mcar_rate0.2  mcar_rate0            0         1    GHK      TRUE   0.7393694
## mcar_rate0.3  mcar_rate0            0         1    GQT      TRUE   0.1046269
## mcar_rate0.4  mcar_rate0            0         1    GQT      TRUE   0.1046269
## mcar_rate0.5  mcar_rate0            0         2    GHK      TRUE   0.6103261
## mcar_rate0.6  mcar_rate0            0         2    GHK      TRUE   0.6103261
##              n_used parameter  estimate    truth     abs_err          err
## mcar_rate0.1     49 Intercept 1.6524332 1.609438 0.042995252  0.042995252
## mcar_rate0.2     49     range 0.3635086 0.300000 0.063508610  0.063508610
## mcar_rate0.3     49 Intercept 1.6255889 1.609438 0.016151028  0.016151028
## mcar_rate0.4     49     range 0.3649102 0.300000 0.064910180  0.064910180
## mcar_rate0.5     49 Intercept 1.2441577 1.609438 0.365280216 -0.365280216
## mcar_rate0.6     49     range 0.2916272 0.300000 0.008372847 -0.008372847
```

Convergence as rate increases

```
conv_tab <- aggregate(
  converged ~ missing_rate + method,
  data = unique(long[, c("scenario_id", "replicate", "method",
                         "missing_rate", "converged")]),
  FUN  = function(x) sum(x) / length(x)
)
names(conv_tab)[3] <- "conv_rate"
conv_tab
```

```
##    missing_rate method conv_rate
## 1           0.0    GHK         1
## 2           0.1    GHK         1
## 3           0.2    GHK         1
```

```
## 4           0.3    GHK         1
## 5           0.5    GHK         1
## 6           0.0    GQT         1
## 7           0.1    GQT         1
## 8           0.2    GQT         1
## 9           0.3    GQT         1
## 10          0.5    GQT         1
```

What to look for: does convergence break down at high missingness rates? If 50% MCAR causes more failures than 10%, that's the first signal that missingness matters.

---

n_used at each rate

Sanity check that masking produced the expected sample size.

```
n_used_tab <- aggregate(
  n_used ~ missing_rate,
  data = long[long$parameter == "Intercept" & long$converged, ],
  FUN  = function(x) c(mean = mean(x), sd = sd(x),
                       min = min(x), max = max(x))
)
n_used_tab
```

```
##   missing_rate n_used.mean n_used.sd n_used.min n_used.max
## 1          0.0          49         0         49         49
## 2          0.1          44         0         44         44
## 3          0.2          39         0         39         39
## 4          0.3          34         0         34         34
## 5          0.5          25         0         25         25
```

At rate 0 should be n=49. At rate 10 should be ~44. At rate 50 should be ~25. If these don't match, the masking logic may have a bug.

MAE vs missingness rate

The headline plot: how does estimation accuracy degrade as data is lost?

```
mae_tab <- aggregate(
  abs_err ~ missing_rate + method + parameter,
  data = long[long$converged, ],
  FUN  = mean
)
names(mae_tab)[4] <- "MAE"

# Split by parameter for readability
mae_intercept <- mae_tab[mae_tab$parameter == "Intercept", ]
mae_range     <- mae_tab[mae_tab$parameter == "range", ]

cat("MAE for Intercept (log lambda):\n")
```

```
## MAE for Intercept (log lambda):
```

```
print(mae_intercept[order(mae_intercept$missing_rate, mae_intercept$method), ],
      row.names = FALSE, digits = 3)
```

```
##  missing_rate method parameter   MAE
##           0.0    GHK Intercept 0.156
##           0.0    GQT Intercept 0.159
##           0.1    GHK Intercept 0.161
##           0.1    GQT Intercept 0.161
##           0.2    GHK Intercept 0.161
##           0.2    GQT Intercept 0.162
##           0.3    GHK Intercept 0.159
##           0.3    GQT Intercept 0.160
##           0.5    GHK Intercept 0.171
##           0.5    GQT Intercept 0.169
```

```
cat("\nMAE for range:\n")
```

```
##
## MAE for range:
```

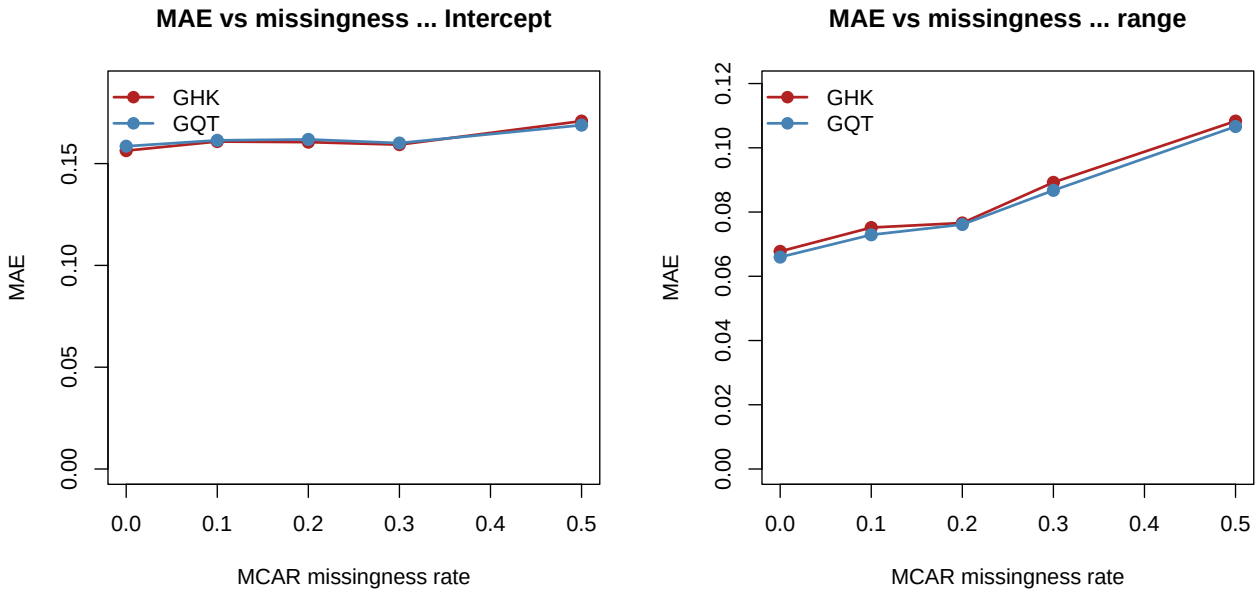
```
print(mae_range[order(mae_range$missing_rate, mae_range$method), ],
      row.names = FALSE, digits = 3)
```

```
##  missing_rate method parameter    MAE
##           0.0    GHK     range 0.0678
##           0.0    GQT     range 0.0660
##           0.1    GHK     range 0.0752
##           0.1    GQT     range 0.0729
##           0.2    GHK     range 0.0766
##           0.2    GQT     range 0.0761
##           0.3    GHK     range 0.0892
##           0.3    GQT     range 0.0867
##           0.5    GHK     range 0.1083
##           0.5    GQT     range 0.1066
```

Figure: MAE vs rate, one panel per parameter

```
par(mfrow = c(1, 2))

for (param in c("Intercept", "range")) {

  d <- mae_tab[mae_tab$parameter == param, ]
  y_max <- max(d$MAE, na.rm = TRUE) * 1.1

  plot(NA, xlim = c(0, 0.5), ylim = c(0, y_max),
       xlab = "MCAR missingness rate",
       ylab = "MAE",
       main = sprintf("MAE vs missingness - %s", param))

  for (m in c("GHK", "GQT")) {
    sub <- d[d$method == m, ]
    sub <- sub[order(sub$missing_rate), ]
    lines(sub$missing_rate, sub$MAE,
          col = ifelse(m == "GHK", "firebrick", "steelblue"),
          lwd = 2)
    points(sub$missing_rate, sub$MAE,
           col = ifelse(m == "GHK", "firebrick", "steelblue"),
           pch = 19, cex = 1.2)
```

```
  }
  legend("topleft", legend = c("GHK", "GQT"),
         col = c("firebrick", "steelblue"),
         lwd = 2, pch = 19, bty = "n")
}
```




```
par(mfrow = c(1, 1))
```

Save the figure

```
dir.create("visualization/missing", recursive = TRUE, showWarnings = FALSE)

pdf("visualization/missing/mae_vs_rate.pdf", width = 10, height = 5)
par(mfrow = c(1, 2))
for (param in c("Intercept", "range")) {
  d <- mae_tab[mae_tab$parameter == param, ]
  y_max <- max(d$MAE, na.rm = TRUE) * 1.1
  plot(NA, xlim = c(0, 0.5), ylim = c(0, y_max),
       xlab = "MCAR missingness rate", ylab = "MAE",
       main = sprintf("MAE vs missingness - %s", param))
  for (m in c("GHK", "GQT")) {
    sub <- d[d$method == m, ]
    sub <- sub[order(sub$missing_rate), ]
    lines(sub$missing_rate, sub$MAE,
          col = ifelse(m == "GHK", "firebrick", "steelblue"), lwd = 2)
    points(sub$missing_rate, sub$MAE,
           col = ifelse(m == "GHK", "firebrick", "steelblue"), pch = 19, cex = 1.2)
  }
  legend("topleft", legend = c("GHK", "GQT"),
         col = c("firebrick", "steelblue"), lwd = 2, pch = 19, bty = "n")
}
par(mfrow = c(1, 1))
dev.off()
```

Bias vs rate

Separate from MAE: does missingness introduce systematic error?

```
bias_tab <- aggregate(
  err ~ missing_rate + method + parameter,
  data = long[long$converged, ],
  FUN  = mean
)
names(bias_tab)[4] <- "bias"

bias_intercept <- bias_tab[bias_tab$parameter == "Intercept", ]
bias_range     <- bias_tab[bias_tab$parameter == "range", ]

cat("Bias for Intercept:\n")
```

```
## Bias for Intercept:
```

```
print(bias_intercept, row.names = FALSE, digits = 3)
```

```
##  missing_rate method parameter    bias
##           0.0    GHK Intercept -0.0183
##           0.1    GHK Intercept -0.0235
##           0.2    GHK Intercept -0.0200
##           0.3    GHK Intercept -0.0133
##           0.5    GHK Intercept -0.0231
##           0.0    GQT Intercept -0.0184
##           0.1    GQT Intercept -0.0225
##           0.2    GQT Intercept -0.0202
##           0.3    GQT Intercept -0.0140
##           0.5    GQT Intercept -0.0209
```

```
cat("\nBias for range:\n")
```

```
##
## Bias for range:
```

```
print(bias_range, row.names = FALSE, digits = 3)
```

```
##  missing_rate method parameter      bias
##           0.0    GHK     range -2.32e-03
##           0.1    GHK     range -5.72e-03
##           0.2    GHK     range -1.85e-03
##           0.3    GHK     range -5.45e-03
##           0.5    GHK     range -5.25e-03
##           0.0    GQT     range  1.92e-03
##           0.1    GQT     range -1.39e-04
##           0.2    GQT     range  1.90e-03
##           0.3    GQT     range -1.84e-05
##           0.5    GQT     range -4.35e-04
```

What to look for: under MCAR, estimators should remain *unbiased* even as variance increases with missingness. If bias grows with the rate, that's a more serious finding than MAE growth (because MAE can increase purely from higher variance).

Runtime as rate increases

Does missingness speed things up? (Fewer data points per fit = fewer likelihood evaluations.)

```
rt <- long[long$converged & long$parameter == "Intercept", ]

rt_tab <- aggregate(
  runtime_sec ~ missing_rate + method,
  data = rt,
  FUN  = mean
)
rt_tab
```

```
##    missing_rate method runtime_sec
## 1           0.0    GHK  0.74228256
## 2           0.1    GHK  0.64156667
## 3           0.2    GHK  0.57010795
## 4           0.3    GHK  0.44726362
## 5           0.5    GHK  0.32711372
## 6           0.0    GQT  0.09322978
## 7           0.1    GQT  0.08128954
## 8           0.2    GQT  0.06658090
## 9           0.3    GQT  0.05639176
## 10          0.5    GQT  0.03584180
```

Summary

```
cat("=== Missingness study summary ===\n\n")
```

```
## === Missingness study summary ===
```

```
cat("Convergence rates by missingness:\n")
```

```
## Convergence rates by missingness:
```

```
print(conv_tab, row.names = FALSE)
```

```
##  missing_rate method conv_rate
##           0.0    GHK         1
##           0.1    GHK         1
##           0.2    GHK         1
##           0.3    GHK         1
##           0.5    GHK         1
##           0.0    GQT         1
##           0.1    GQT         1
##           0.2    GQT         1
##           0.3    GQT         1
##           0.5    GQT         1
```

```
cat("\nRelative MAE increase from 0% to 50% missing:\n")
```

```
##
## Relative MAE increase from 0% to 50% missing:
```

```
for (param in c("Intercept", "range")) {
  for (m in c("GHK", "GQT")) {
```

```
    base_mae <- mae_tab$MAE[mae_tab$missing_rate == 0 &
                            mae_tab$method == m &
                            mae_tab$parameter == param]
    max_mae <- mae_tab$MAE[mae_tab$missing_rate == 0.5 &
                           mae_tab$method == m &
                           mae_tab$parameter == param]
    if (length(base_mae) == 1 && length(max_mae) == 1) {
      cat(sprintf("  %s / %s: baseline MAE=%.4f  -> 50%% MAE=%.4f  (%.1fx)\n",
                  param, m, base_mae, max_mae, max_mae / base_mae))
    }
  }
}
```

```
##   Intercept / GHK: baseline MAE=0.1564  -> 50% MAE=0.1709  (1.1x)
##   Intercept / GQT: baseline MAE=0.1585  -> 50% MAE=0.1689  (1.1x)
##   range / GHK: baseline MAE=0.0678  -> 50% MAE=0.1083  (1.6x)
##   range / GQT: baseline MAE=0.0660  -> 50% MAE=0.1066  (1.6x)
```

Likely findings to include in the report

Fill in as numbers come out:

1. **Convergence.** Both methods maintain _____% convergence even at 50% missingness.
2. **Bias.** Remains near zero across all rates (expected under MCAR).
3. **Accuracy degradation.** MAE grows approximately linearly with missingness rate. At 50%, MAE is roughly _____x the baseline.
4. **Method comparison.** GHK and GQT degrade at similar rates — no evidence that one method is more robust to MCAR than the other.
5. **Runtime.** Runtime decreases as rate increases, due to smaller effective sample size.

End of missingness analysis